41

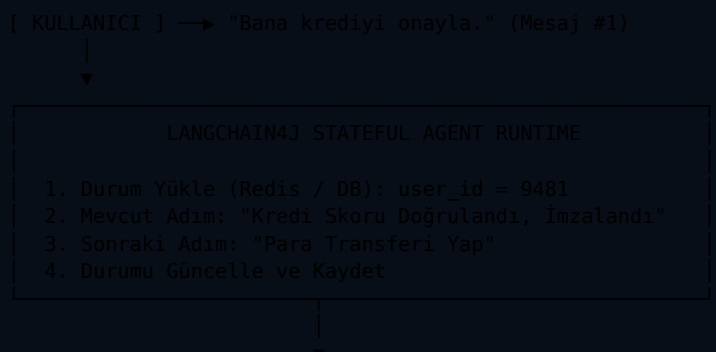
BÖLÜM: AI AJANLARI (AI AGENTS), OTONOM AKIŞLAR VE BELLEK MIMARISI | MODULE: AI AGENTS, AUTONOMOUS LOOPS & CONVERSATIONAL MEMORY SYSTEMS

LangChain4j ile Durumsal (Stateful) Ajan Tasarımı

Building Stateful Agents with LangChain4j: State Persistence & Resumption

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Akıllı bir veznedarın önüne her gittiğinizde size 'Sen kimdin? Ne istiyordun?' diye sormadığını düşünün. Masasında sizin dosyanız açıktır; 'En son kimlik fotokopinizi almıştık, şimdi imzanızı atın' der (Stateful / Durumsal). LangChain4j'in durumsal ajanları böyledir: Kullanıcı tarayıcıyı kapatsa, aradan 3 gün geçse bile sistem kullanıcının nerede kaldığını hatırlar. Ajan hafızasını kaybetmez!

🇬🇧 ENGLISH (ELI5)

Imagine a bank teller who suffers complete amnesia every 30 seconds, forcing you to explain your loan application from scratch every time you speak. A stateful teller keeps your physical case file open on the desk: 'We already verified your tax returns yesterday; today we simply need your signature.' LangChain4j's stateful agents maintain persistent lifecycle states across sessions and machine restarts: an agent pauses, saves state to Redis, and resumes seamlessly days later!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Stateful Ajan Mühendisliği: 1) State Entity: Ajanın anlık değişkenlerini, tamamlanan adımlarını ve ara çıktılarını tutan Java Record modelidir. 2) Checkpointing: Her tool çağrısından sonra ajan durumu veritabanına (PostgreSQL / Redis) snapshot olarak yazılır. 3) Crash Recovery (Çökme Kurtarma): Sunucu o anda çökse veya pod yeniden başlasa bile, yeni pod son checkpoint'ten durumu yükler ve ajanı sıfırdan başlatmadan kaldığı adımdan devam ettirir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Stateful Agent Architecture & Checkpointing: 1) State Machine Semantics: Models agent execution as an explicit Finite State Machine (FSM) where state transitions are triggered by tool outputs or human approvals. 2) Durable Checkpointing: After every state change, an immutable state snapshot is persisted to an ACID store (PostgreSQL or Redis JSON). 3) Crash Fault-Tolerance: If a container restarts or encounters an Out-of-Memory (OOM) error mid-task, the new pod hydrates state from the latest checkpoint, resuming execution without duplicate side-effects.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & LangChain4j: Durumsal Ajan Durum (State) Modeli
import java.time.Instant;
import java.util.Map;

public class AgentStateSnapshot {
    public record AgentState(
        String conversationId,
        String currentStage, // INIT, COLLECTING_INFO, AWAITING_APPROVAL, COMPLETED
        Map<String, Object> contextVariables,
        Instant lastUpdated
    ) {}

    public static AgentState advanceStage(AgentState previous, String newStage) {
        return new AgentState(
            previous.conversationId(),
            newStage,
            previous.contextVariables(),
            Instant.now()
        );
    }
}

void main() {
    var state = new AgentStateSnapshot.AgentState("conv_101", "INIT", Map.of("userId", "usr_99"),
Instant.now());
    IO.println("Ajan Başlangıç Durumu: " + state.currentStage());

    var updated = AgentStateSnapshot.advanceStage(state, "AWAITING_APPROVAL");
    IO.println("Ajan Yeni Durumu: " + updated.currentStage() + " | Güncellenme: " +
updated.lastUpdated());
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Durum Şişmesi (State Bloat): Ajanın her mesajındaki tüm verileri sonsuza kadar `contextVariables` içinde saklamak Redis'i şişirir. Görev tamamlandığında (COMPLETED) geçici durum değişkenleri temizlenmeli (TTL ile temizlik), sadece nihai özet arşivlenmelidir.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

State bloat hazard: Accumulating megabytes of unpruned intermediate tool outputs inside state objects exhausts Redis memory. Implement explicit state lifecycle management: set Redis TTLs and prune transient execution payloads upon task completion.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Durumsal (Stateful) ajanlar, uzun süren iş akışlarında oturumlar arası hafıza sağlar.
- Checkpointing, sunucu çökmelerinde ajanın kaldığı yerden devam etmesini garanti eder.
- Tamamlanan durumlar bellek şişmesini önlemek için temizlenmeli veya arşivlenmelidir.

🎯 Key Takeaways for the Interview

- Stateful agents preserve workflow context across multi-session, long-running processes.
- Checkpointing provides fault tolerance, enabling seamless resumption after server restarts.
- Completed agent states must be pruned or archived to maintain lean memory profiles.